



Rapport de Projet

Projet Vision Artificielle et traitements des Images

8INF883

Sujet : Développement de modèles de génération de visages (CVAE / CGAN).

Cédric GAUTHERET et Romain AMIGON

25 avril 2026

Table des matières

1	Introduction	2
1.1	Sujet	2
1.2	Motivations	2
2	Les données	3
2.1	Exploration	4
2.2	Pré-traitements	5
3	Le modèle CGAN	7
3.1	Le modèle	7
3.2	Entraînements et résultats	11
3.2.1	Entraînement 5 époques (premiers tests)	12
3.2.2	LPIPS	13
3.2.3	Entraînement 30 époques	14
3.2.4	Entraînement 15 époques (modèle final)	15
3.2.5	Pistes d'améliorations	17
4	Le modèle CVAE	18
4.1	Résultats CVAE	20
4.1.1	Reconstruction images dataset test	20
4.1.2	ajout features images dataset train	20
4.1.3	Génération aléatoire avec feature	21
5	Modèle exploratoire : CVAE-Transformer (ViT-CVAE)	23
5.1	Résultats visuels et limites	23
6	Interface pour notre application (Inférence)	25
7	Conclusion	26
8	Bibliographie	27

1 Introduction

1.1 Sujet

L'objectif de notre projet est de faire de la génération d'image conditionnelle. On veut générer des visages comportant des attributs que l'on peut choisir selon une liste, par exemple on peut générer un visage de femme avec des lunettes et un chapeau.

1.2 Motivations

Ce projet peut s'inscrire dans une idée d'application, par exemple pour les opticiens, coiffeurs ou pour des marchands de la mode.

En effet, on peut imaginer une application sur téléphone avec laquelle l'utilisateur peut prendre son visage et y ajouter la génération de lunette ou changer sa couleur de cheveux pour voir comment cela rend et se projeter avant de faire un achat.

Ce projet sera l'occasion d'explorer les CGAN/DCGAN et CVAE.

On utilisera pour ce projet le jeu de données **CelebA**.

On va dans la suite de ce rapport, commenter certains éléments de code pour mieux comprendre nos implémentations, évidemment ce sera fait de manière non exhaustive pour ne pas le surcharger.

2 Les données

Le jeu de données (dataset) **CelebFaces Attributes (CelebA)** est un jeu de données à grande échelle d'attributs faciaux contenant plus de 200000 images de célébrités.

La grande force de cette base de données réside dans sa diversité : elle intègre d'importantes variations de poses, des arrière-plans complexes et désordonnés, ainsi qu'une grande variété de profils humains. Le tout est soutenu par un volume massif d'images et des annotations d'une grande richesse. *(Ces données ont été initialement collectées par les chercheurs du MMLAB de l'Université chinoise de Hong Kong).*

Les caractéristiques principales de ce jeu de données sont les suivantes :

- Volume : 202599 images de visages.
- Identités : 10177 célébrités uniques.
- Annotations : 40 attributs binaires par image (ex. : *Souriant, Lunettes, Homme, Jeune*).
- Repères (Landmarks) : 5 points de repère par image (yeux, nez, coins de la bouche).

CelebA est une référence incontournable en vision par ordinateur pour l'entraînement des réseaux antagonistes génératifs (GAN). Sa grande diversité de poses, d'arrière-plans et de variations faciales le rend idéal pour générer des visages très réalistes.

Le jeu de données se présente sous la forme suivante une fois téléchargé :

```
CelebA/
├── img_align_celeba/          ... Répertoire principal des images
│   └── img_align_celeba/      ... Sous-dossier contenant les fichiers .jpg
├── list_attr_celeba.csv        ... Attributs binaires (sourire, lunettes,
│                               etc.)
├── list_bbox_celeba.csv        ... Coordonnées des boîtes englobantes
├── list_eval_partition.csv     ... Répartition (train, valid, test)
└── list_landmarks_align_celeba.csv ... Points de repère faciaux (yeux,
    nez, bouche)
```

FIGURE 1 – Structure de l'arborescence du dataset CelebA

img_align_celeba : Le coeur du dataset. Il contient toutes les images de visages, qui ont déjà été recadrées et alignées (les yeux sont toujours à peu près au même niveau).

list_eval_partition.csv : Ce fichier propose une répartition standardisée des images pour l'apprentissage :

- Train : Images 1 à 162770.
- Validation : Images 162771 à 182637.
- Test : Images 182638 à 202599.

Étant donné que l'on ne va pas faire de la classification mais de la génération d'image, on n'a pas besoin d'utiliser une telle répartition.

list_bbox_celeba.csv : Contient les informations des *bounding boxes* permettant de localiser précisément le visage dans l'image d'origine. Les colonnes **x_1** et **y_1** indiquent les coordonnées du coin supérieur gauche de la boîte, tandis que **width** et **height** en définissent la largeur et la hauteur.

list_landmarks_align_celeba.csv : Fournit les coordonnées exactes (x, y) de 5 points clés du visage : l'oeil gauche, l'oeil droit, le nez, ainsi que les coins gauche et droit de la bouche.

list_attr_celeba.csv : Le fichier des *labels*. Il répertorie les 40 attributs de chaque image. La valeur **1** indique que l'attribut est présent (positif), tandis que la valeur **-1** indique son absence (négatif).

On a choisi ce dataset pour ses **40 attributs** qui permettent de conditionner un **CGAN**. Au lieu de générer des visages de manière purement aléatoire, nous pourrions utiliser ces étiquettes pour diriger le modèle et générer des visages "à la carte". Par exemple, on pourra demander spécifiquement au générateur de créer "un homme blond avec des lunettes" ou "une femme souriante".

Pour notre **CGAN** basé sur des réseaux convolutifs (**CNN**), nous devrions pré-traiter ces images. Nous allons les recadrer, les redimensionner à une taille standard et uniforme (ex. : 64x64), et normaliser la valeur de leurs pixels dans la plage [-1, 1] afin de correspondre à la fonction d'activation Tanh habituellement présente à la sortie du Générateur.

2.1 Exploration

On peut afficher les premières images pour avoir une idée de ce à quoi elles ressemblent dans le jeu de données.

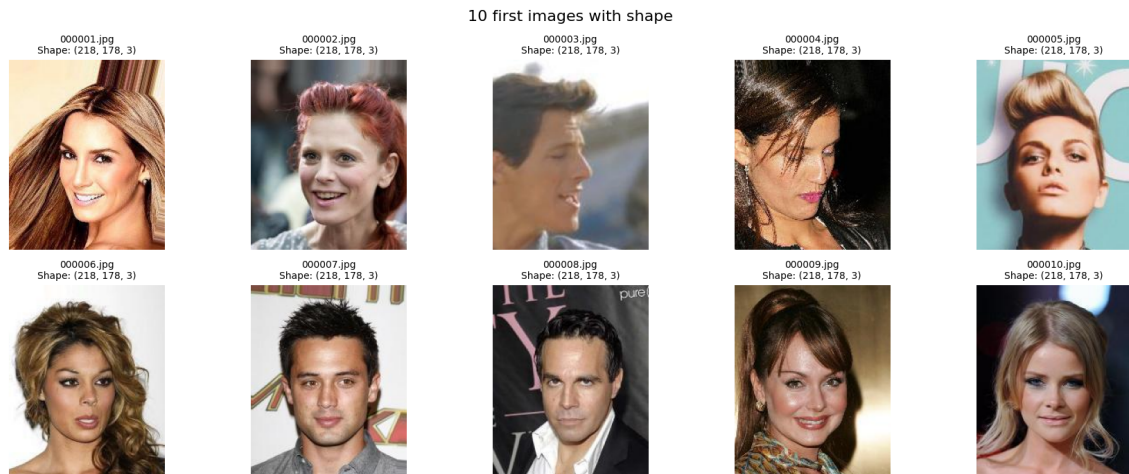


FIGURE 2 – Les 10 premières images et leur taille

On remarque que toutes les images ont la même taille : `(218, 178, 3)`. Pour s'en assurer, on utilise le script suivant :

```
1 unique_shapes = set()
2
3 for i, img_name in enumerate(img_names):
4     img_path = os.path.join(img_dir, img_name)
5
6     with Image.open(img_path) as img:
7         channels = 3 if img.mode == 'RGB' else (1 if img.mode == 'L' else img.mode)
8         shape = (img.size[1], img.size[0], channels)
9
10        unique_shapes.add(shape)
11
12 print(f"Number of unique shapes found: {len(unique_shapes)}")
13
14 for shape in unique_shapes:
15     print(f"- {shape}")
```

Listing 1 – Vérification shape des images

On vérifie donc que toutes les images ont la même taille : `218x178`. Pour autant, on appliquera un redimensionnement dans notre pré-traitement pour diminuer la taille des images.

On peut aussi afficher la fréquence des attributs dans le jeu de données pour avoir une idée de la répartition (les images pouvant avoir plusieurs attributs).

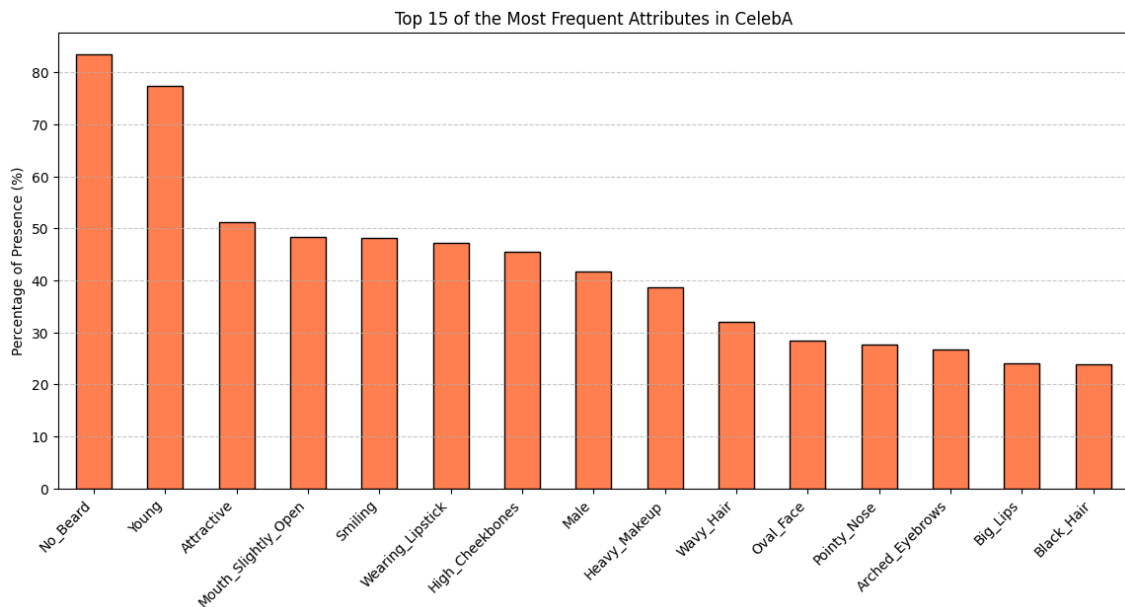


FIGURE 3 – Fréquence des attributs

On peut ainsi afficher les attributs qui sont les plus rares.

```

1 Most Rare Attributes :
2 Double_Chin      4.668829
3 Pale_Skin        4.294690
4 Gray_Hair        4.194986
5 Mustache         4.154512
6 Bald            2.244335

```

On remarque alors que la répartition est très inégale, certains attributs sont sur-représentés (proportionnellement) comme `No_Beard` ou `Young` et d'autres sont sous-représentés et seront donc sûrement non utilisables comme `Bald` ou `Mustache`.

On retiendra par exemple : `Smiling`, `Male`, `No_Beard`, `Wearing_Lipstick`, `Eyeglasses`.

Par ailleurs, on récupère la liste des 40 attributs

```

1 List of 40 unique attributes :
2
3 5_o_Clock_Shadow, Arched_Eyebrows, Attractive, Bags_Under_Eyes, Bald
4 Bangs, Big_Lips, Big_Nose, Black_Hair, Blond_Hair
5 Blurry, Brown_Hair, Bushy_Eyebrows, Chubby, Double_Chin
6 Eyeglasses, Goatee, Gray_Hair, Heavy_Makeup, High_Cheekbones
7 Male, Mouth_Slightly_Open, Mustache, Narrow_Eyes, No_Beard
8 Oval_Face, Pale_Skin, Pointy_Nose, Receding_Hairline, Rosy_Cheeks
9 Sideburns, Smiling, Straight_Hair, Wavy_Hair, Wearing_Earrings
10 Wearing_Hat, Wearing_Lipstick, Wearing_Necklace, Wearing_Necktie, Young

```

2.2 Pré-traitements

Avant d'appliquer la normalisation, la fonction `transforms.ToTensor()` convertit l'image brute (dont les pixels ont des valeurs entières comprises entre 0 et 255) en un tenseur PyTorch où les valeurs sont redimensionnées en nombres flottants compris entre 0.0 et 1.0.

Nous allons appliquer une normalisation avec une moyenne de 0.5 et un écart-type de 0.5.

Pour un pixel noir (0.0) : $(0.0 - 0.5) / 0.5 = -1.0$

Pour un pixel blanc (1.0) : $(1.0 - 0.5) / 0.5 = 1.0$

Toutes les valeurs de nos images réelles sont désormais étalées de manière symétrique sur la plage $[-1, 1]$.

Dans l'architecture classique d'un GAN, la toute dernière couche du **Générateur** utilise une fonction d'activation **Tanh**.

La propriété mathématique de la fonction **Tanh** est de sortir exclusivement des valeurs comprises entre -1 et 1.

Pour que le **Discriminateur** puisse faire son travail correctement et comparer de manière équitable les "fausses" images créées par le **Générateur** et les "vraies" images du dataset CelebA, on doit utiliser les mêmes échelles.

Les chercheurs ont en effet prouvé (notamment dans le papier de recherche fondateur du DCGAN [1]) que l'utilisation de la plage $[-1, 1]$ avec une activation Tanh aide le modèle à converger plus rapidement et de manière plus stable. Cela génère des images avec des contrastes plus forts et réduit le risque de disparition du gradient (*vanishing gradient*) pendant l'entraînement par rapport à une plage $[0, 1]$.

```
1 transform = transforms.Compose([
2     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
3     transforms.CenterCrop(IMAGE_SIZE),
4     transforms.ToTensor(),
5     transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))
6 ])
7
8 class CelebADataset(Dataset):
9     def __init__(self, root_dir, attr_df, transform=None):
10         self.root_dir = root_dir
11         self.image_files = [f for f in os.listdir(root_dir) if f.endswith('.jpg')]
12         self.attr_df = attr_df
13         self.transform = transform
14
15     def __len__(self):
16         return len(self.image_files)
17
18     def __getitem__(self, idx):
19         img_name = self.image_files[idx]
20         img_path = os.path.join(self.root_dir, img_name)
21
22         image = Image.open(img_path).convert('RGB')
23         if self.transform:
24             image = self.transform(image)
25
26         attributes = torch.tensor(self.attr_df.loc[img_name].values, dtype=torch.
float32)
27
28         return image, attributes
29
30 celeba_dataset = CelebADataset(root_dir=img_dir, attr_df=df_attr, transform=
transform)
31
32 dataloader = DataLoader(
33     celeba_dataset,
34     batch_size=BATCH_SIZE,
35     shuffle=True,
36     num_workers=0,
37     pin_memory=True
38 )
```

Listing 2 – Chargement du dataset dans un `Dataloader` PyTorch

3 Le modèle CGAN

Le processus de création du GAN est documenté et disponible dans le notebook jupyter suivant : GAN/GAN.ipynb

3.1 Le modèle

Un réseau antagoniste génératif classique repose sur une compétition féroce entre deux modèles de neurones.

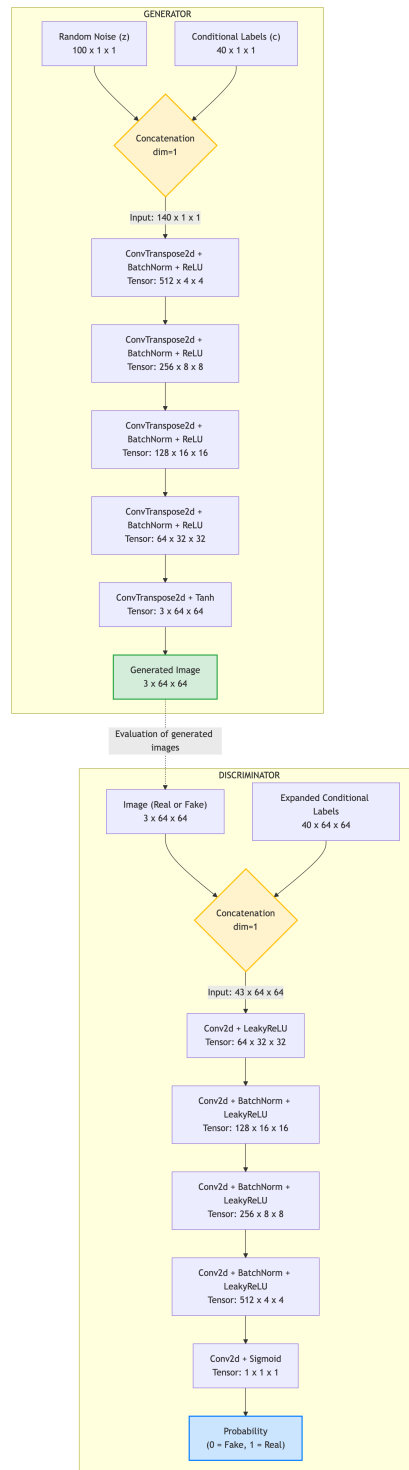


FIGURE 4 – Architecture du CGAN

Le premier est le **Générateur**, dont le but est de transformer un vecteur de bruit aléatoire abstrait, noté z , en une donnée réaliste, comme une image. Le second est le **Discriminateur**, qui agit comme un détective cherchant à séparer les véritables images provenant de la base de données de celles fabriquées par le Générateur.

Dans un Conditional GAN, ou **CGAN**, cette dynamique est enrichie par l’ajout d’une information supplémentaire, notée c , qui représente une condition. Au lieu de générer des visages au hasard, le modèle apprend à lier des caractéristiques visuelles spécifiques à cette variable c . Mathématiquement, cette compétition se modélise par une fonction objectif **minimax** où le **Générateur** tente de minimiser la fonction que le Discriminateur cherche à maximiser.

L’équation fondamentale du CGAN s’écrit donc

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|c)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|c)|c))]$$

Pour arbitrer ce jeu, nous utilisons la Binary Cross Entropy Loss, ou **BCELoss**.

Étant donné que le rôle final du **Discriminateur** est de déterminer si une image est vraie ou fausse, le problème se résume à une classification binaire classique. La fonction Sigmoid placée à la toute fin du réseau du **Discriminateur** comprime la sortie pour fournir une probabilité continue comprise entre 0 et 1. La **BCELoss** va alors mesurer la distance mathématique entre cette probabilité prédite et la véritable étiquette de l’image.

Soit,

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

où y_i représente l’étiquette réelle, valant 1 pour une vraie image et 0 pour une fausse, tandis que $\hat{y}_i$ représente la prédiction du modèle.

Lors de l’entraînement, le **Discriminateur** cherche à minimiser cette erreur par rapport à la réalité, tandis que le Générateur calcule sa propre perte en passant ses fausses images au **Discriminateur** mais en cherchant à minimiser l’erreur par rapport à l’étiquette cible 1, forçant ainsi le modèle à s’améliorer pour tromper son adversaire.

Dans notre implémentation, le Générateur est conçu selon les principes de l’architecture **DCGAN** ([1]), basée sur des convolutions profondes. Tout commence par la fusion de nos entrées. Nous prenons notre vecteur de bruit de dimension 100 et le concaténons directement avec notre vecteur de conditions de dimension 40. Le réseau démarre donc avec un volume d’entrée de dimension $140 \times 1 \times 1$. Pour transformer ce minuscule bloc d’informations en une image complète, nous utilisons des couches de convolutions transposées. Ces couches agissent à l’inverse des convolutions classiques en effectuant un suréchantillonnage spatial. À chaque étape, la résolution de notre image en devenir est doublée, passant de 4×4 , puis 8×8 , jusqu’à atteindre notre objectif de 64×64 pixels.

Après presque chaque convolution, nous appliquons une normalisation par lot, ou *BatchNorm*, qui standardise les activations internes et empêche le modèle de diverger de manière chaotique lors des premières phases d’apprentissage. La non-linéarité est assurée par la fonction ReLU qui accélère la convergence. Enfin, la toute dernière couche utilise une activation Tanh pour forcer la sortie mathématique à se situer strictement dans l’intervalle $[-1, 1]$, correspondant exactement à l’échelle de normalisation que nous avons appliquée à nos images réelles lors du prétraitement.

Le **Discriminateur** effectue exactement le chemin inverse. Puisqu’il est impossible de concaténer simplement un vecteur unidimensionnel de taille 40 avec une image bidimensionnelle de taille 64×64 , l’architecture commence par répéter le vecteur conditionnel sur toute la surface de l’image. Cela crée 40 nouveaux canaux d’information, empilés virtuellement derrière les 3 canaux de couleur RGB de l’image. La première couche de convolution reçoit donc une pile de données de 43 canaux. Le réseau utilise ensuite une série de convolutions standards avec un pas, ou *stride*, de 2.

Ce choix architectural volontaire remplace les traditionnelles opérations de *MaxPooling*, laissant au réseau la liberté d’apprendre par lui-même la meilleure méthode pour réduire la résolution spatiale à chaque étape. Une caractéristique vitale de ce **Discriminateur** est l’utilisation exclusive de l’activation *LeakyReLU* avec une pente négative de 0.2. Contrairement au ReLU standard qui met purement et simplement à zéro toutes les valeurs négatives, le LeakyReLU laisse passer un infime flux d’information. Cette astuce empêche le **Discriminateur** de bloquer complètement la circulation des gradients, garantissant que le Générateur reçoive toujours un retour d’information mathématique pour corriger ses erreurs, même lorsque ses fausses images sont facilement démasquées au début de l’entraînement.

Le Générateur

Pour que le modèle prenne en compte les 40 attributs faciaux (c), le Générateur intègre l'information conditionnelle dès l'entrée du réseau : le vecteur de bruit aléatoire $z \in \mathbb{R}^{100}$ est concaténé avec le vecteur d'attributs $c \in \mathbb{R}^{40}$, formant ainsi un vecteur global de taille 140. L'objectif est de transformer ce vecteur latent en une image complexe de 64×64 pixels. Pour ce faire, le réseau utilise des couches de convolutions transposées (`nn.ConvTranspose2d()`) qui augmentent la résolution spatiale par étapes successives (de 4×4 jusqu'à 64×64). Chaque couche est suivie d'une normalisation par lot (`nn.BatchNorm2d()`) afin de standardiser les activations intermédiaires, stabilisant ainsi l'apprentissage et facilitant la circulation des gradients. Alors que les couches cachées utilisent l'activation `nn.ReLU()` pour introduire des non-linéarités sans saturer, la couche de sortie emploie la fonction tangente hyperbolique (`nn.Tanh()`). Ce choix contraint les pixels générés dans la plage $[-1, 1]$, assurant une cohérence parfaite avec le prétraitement appliqué aux images réelles.

Le choix de la profondeur des réseaux et de l'évolution des dimensions des *feature maps* repose sur un équilibre entre complexité et stabilité de convergence. Pour une résolution cible de 64×64 pixels, une architecture à cinq blocs de convolution est standard : elle permet une réduction (ou augmentation) progressive par puissances de deux ($2^2 \rightarrow 2^6$). Dans le Générateur, le choix de débiter avec `FEATURES_GEN` $\times 8$ (512 canaux) pour finir à 3 canaux permet de condenser l'information sémantique complexe des 140 variables d'entrée dans un grand nombre de filtres initiaux, avant de les projeter vers une structure spatiale plus large mais moins profonde. Cette décompression progressive est essentielle pour que le réseau puisse d'abord générer des formes globales cohérentes avant de s'affiner sur les détails texturaux.

```

1 class Generator(nn.Module):
2     def __init__(self, z_dim, c_dim, features_g):
3         super(Generator, self).__init__()
4         # Input to the first layer is noise (z_dim) + condition (c_dim)
5         in_channels = z_dim + c_dim
6
7         self.gen = nn.Sequential(
8             # Input: (in_channels) x 1 x 1
9             nn.ConvTranspose2d(in_channels, features_g * 8, kernel_size=4, stride
10                                =1, padding=0, bias=False),
11             nn.BatchNorm2d(features_g * 8),
12             nn.ReLU(True),
13
14             # State: (features_g*8) x 4 x 4
15             nn.ConvTranspose2d(features_g * 8, features_g * 4, kernel_size=4,
16                                stride=2, padding=1, bias=False),
17             nn.BatchNorm2d(features_g * 4),
18             nn.ReLU(True),
19
20             # State: (features_g*4) x 8 x 8
21             nn.ConvTranspose2d(features_g * 4, features_g * 2, kernel_size=4,
22                                stride=2, padding=1, bias=False),
23             nn.BatchNorm2d(features_g * 2),
24             nn.ReLU(True),
25
26             # State: (features_g*2) x 16 x 16
27             nn.ConvTranspose2d(features_g * 2, features_g, kernel_size=4, stride=2,
28                                padding=1, bias=False),
29             nn.BatchNorm2d(features_g),
30             nn.ReLU(True),
31
32             # State: (features_g) x 32 x 32
33             nn.ConvTranspose2d(features_g, CHANNELS, kernel_size=4, stride=2,
34                                padding=1, bias=False),
35             # Output uses Tanh to map values to [-1, 1]
36             nn.Tanh(),
37             # Final output: 3 x 64 x 64
38         )
39
40     def forward(self, x, labels):
41         # Reshape labels to append them to the latent noise vector
42         labels = labels.unsqueeze(2).unsqueeze(3) # Shape: (batch, C_DIM, 1, 1)
43         # Concatenate noise and labels along the channel dimension
44         x = torch.cat([x, labels], dim=1)
45         return self.gen(x)

```

Listing 3 – Architecture du Générateur

Le Discriminateur

Le Discriminateur agit comme un classifieur binaire dont l'entrée est hybride : il doit analyser à la fois la structure de l'image et sa correspondance avec les attributs. Le vecteur c est "étiré" spatialement pour créer 40 canaux (calques) à la taille de l'image (64×64). Le réseau traite donc un bloc de données composé de 3 canaux de couleurs (RGB) et de 40 canaux de conditions, soit un total de 43 canaux en entrée. Le sous-échantillonnage est effectué via des convolutions avec un pas de deux (`stride=2`), permettant au modèle d'apprendre sa propre réduction de dimension plutôt que d'utiliser un *MaxPooling* figé. Conformément aux recommandations DCGAN ([1]), la première couche ne comporte pas de `nn.BatchNorm()` pour préserver la distribution brute des données. L'activation `nn.LeakyReLU()` avec une pente de 0.2 est systématiquement privilégiée pour éviter le problème des "neurones morts", tandis qu'une `nn.Sigmoid()` finale fournit la probabilité d'authenticité. L'ensemble est entraîné avec l'optimiseur Adam ($lr = 0.0002$, $\beta_1 = 0.5$), des paramètres hyper-spécifiques cruciaux pour prévenir les instabilités et les oscillations communes lors de l'entraînement des GANs.

Côté Discriminateur, la structure symétrique commençant à `FEATURES_DISC` (64 canaux) permet de capturer d'abord des motifs locaux simples (bords, couleurs), puis d'augmenter la profondeur à chaque réduction spatiale pour extraire des caractéristiques de plus en plus abstraites et globales. L'utilisation d'une base de 64 filtres (`FEATURES_GEN/DISC`) offre un compromis optimal : elle fournit suffisamment de paramètres pour modéliser la diversité du dataset CelebA (variété de visages, poses et éclairages) sans pour autant atteindre une complexité de calcul qui rendrait l'entraînement prohibitif ou sujet à un surapprentissage rapide. Ce dimensionnement garantit que le Discriminateur est assez puissant pour guider le Générateur sans devenir "trop fort" trop rapidement, évitant ainsi la disparition du gradient indispensable à la mise à jour des poids du Générateur.

```
1 class Discriminator(nn.Module):
2     def __init__(self, c_dim, features_d):
3         super(Discriminator, self).__init__()
4         # Input channels = image channels (3) + condition channels (40)
5         in_channels = CHANNELS + c_dim
6
7         self.disc = nn.Sequential(
8             # Input: (in_channels) x 64 x 64
9             nn.Conv2d(in_channels, features_d, kernel_size=4, stride=2, padding=1,
10             bias=False),
11             nn.LeakyReLU(0.2, inplace=True),
12
13             # State: (features_d) x 32 x 32
14             nn.Conv2d(features_d, features_d * 2, kernel_size=4, stride=2, padding
15             =1, bias=False),
16             nn.BatchNorm2d(features_d * 2),
17             nn.LeakyReLU(0.2, inplace=True),
18
19             # State: (features_d*2) x 16 x 16
20             nn.Conv2d(features_d * 2, features_d * 4, kernel_size=4, stride=2,
21             padding=1, bias=False),
22             nn.BatchNorm2d(features_d * 4),
23             nn.LeakyReLU(0.2, inplace=True),
24
25             # State: (features_d*4) x 8 x 8
26             nn.Conv2d(features_d * 4, features_d * 8, kernel_size=4, stride=2,
27             padding=1, bias=False),
28             nn.BatchNorm2d(features_d * 8),
29             nn.LeakyReLU(0.2, inplace=True),
30
31             # State: (features_d*8) x 4 x 4
32             nn.Conv2d(features_d * 8, 1, kernel_size=4, stride=1, padding=0, bias=
33             False),
34             nn.Sigmoid() # Output a probability [0, 1]
35         )
36
37     def forward(self, x, labels):
38         # Expand labels to match the image spatial dimensions (64x64)
39         labels = labels.unsqueeze(2).unsqueeze(3)
40         labels = labels.repeat(1, 1, x.size(2), x.size(3)) # Shape: (batch, C_DIM,
41         64, 64)
42         # Concatenate image and expanded labels
43         x = torch.cat([x, labels], dim=1)
```

```
38         return self.disc(x)
```

Listing 4 – Architecture du Discriminateur

3.2 Entraînements et résultats

On définit la boucle d'entraînement suivante :

```
1 def train_cgan(gen, disc, dataloader, device, num_epochs=5, z_dim=100):
2     """
3     Trains the Conditional GAN and returns the loss history.
4     """
5     # optimizers
6     opt_gen = optim.Adam(gen.parameters(), lr=0.0002, betas=(0.5, 0.999))
7     opt_disc = optim.Adam(disc.parameters(), lr=0.0002, betas=(0.5, 0.999))
8
9     # loss function
10    criterion = nn.BCELoss()
11
12    # lists to keep track of progress
13    G_losses = []
14    D_losses = []
15
16    for epoch in range(num_epochs):
17        for batch_idx, (real_images, labels) in enumerate(dataloader):
18
19            real_images = real_images.to(device)
20            labels = labels.to(device)
21            batch_size = real_images.shape[0]
22
23            # Targets for BCE Loss (1 for real, 0 for fake)
24            # we use a slight label smoothing (0.9 instead of 1.0) for real images
25            to stabilize training
26            real_targets = torch.full((batch_size, 1, 1, 1), 0.9, device=device)
27            fake_targets = torch.zeros((batch_size, 1, 1, 1), device=device)
28
29            # =====
30            # Train Discriminator: Maximize log(D(x)) + log(1 - D(G(z)))
31            # =====
32            disc.zero_grad()
33
34            # loss on real images
35            output_real = disc(real_images, labels)
36            loss_disc_real = criterion(output_real, real_targets)
37
38            # loss on fake images
39            # generate random noise
40            noise = torch.randn(batch_size, z_dim, 1, 1, device=device)
41            # generate fake images conditioned on the real labels
42            fake_images = gen(noise, labels)
43
44            output_fake = disc(fake_images.detach(), labels) # detach() prevents
45            gradients from flowing into Generator
46            loss_disc_fake = criterion(output_fake, fake_targets)
47
48            # total Discriminator loss and backprop
49            loss_disc = (loss_disc_real + loss_disc_fake) / 2
50            loss_disc.backward()
51            opt_disc.step()
52
53            # =====
54            # Train Generator: Maximize log(D(G(z)))
55            # =====
56            gen.zero_grad()
57
58            # we want the discriminator to believe our fake images are real (target
59            = 1)
60            output_fake_for_gen = disc(fake_images, labels)
61
62            loss_gen = criterion(output_fake_for_gen, torch.ones_like(
63            output_fake_for_gen))
64            loss_gen.backward()
65            opt_gen.step()
66
67            # save losses for plotting later
```

```

64         G_losses.append(loss_gen.item())
65         D_losses.append(loss_disc.item())
66
67     # return the recorded losses
68     return G_losses, D_losses

```

Listing 5 – Entraînement du CGAN

3.2.1 Entraînement 5 époques (premiers tests)

On va faire un premier entraînement sur 5 époques pour tester notre architecture et valider le modèle.

```

1 gen_5 = Generator(Z_DIM, C_DIM, FEATURES_GEN).to(device)
2 disc_5 = Discriminator(C_DIM, FEATURES_DISC).to(device)
3
4 NUM_EPOCHS = 5
5
6 G_losses_5, D_losses_5 = train_cgan(gen_5, disc_5, dataloader, device, num_epochs=
    NUM_EPOCHS, z_dim=Z_DIM)

```

Listing 6 – Entraînement du CGAN sur 5 époques

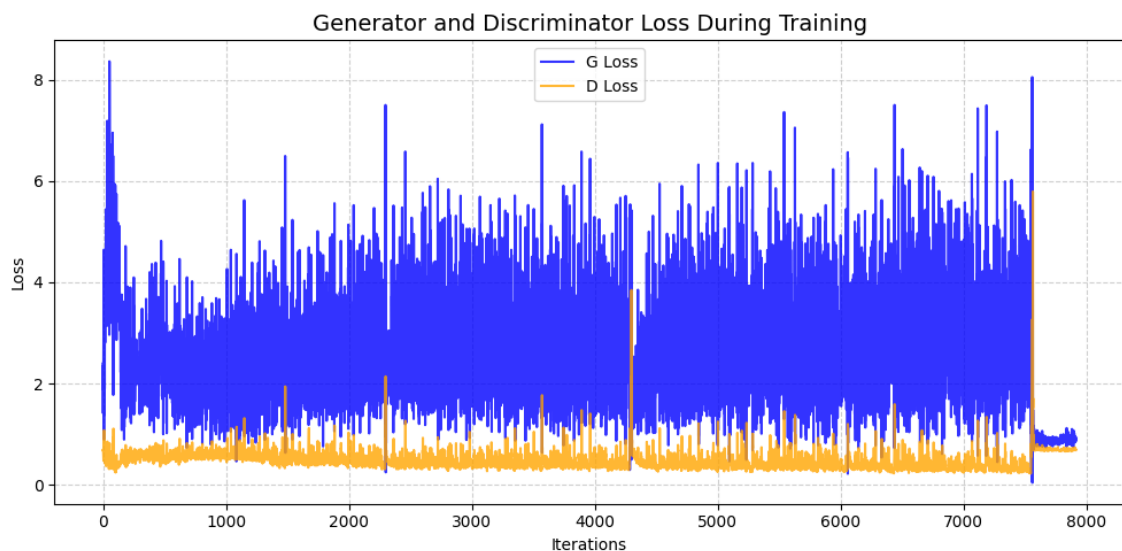


FIGURE 5 – Courbes de Loss du CGAN sur 5 époques

Il faudrait utiliser une échelle log pour bien visualiser les courbes mais la différence d'échelle entre le générateur et le discriminateur est telle que l'on ne verrait rien.

De plus, on représente en abscisses le nombre de batch et pas époques pour plus de précision et lisibilité (mais on a en gros une époque tous les 1500 batch car `batch_size = 128`).

On remarque que la loss commence à se stabiliser vers la fin de l'entraînement ce qui est un bon signe.

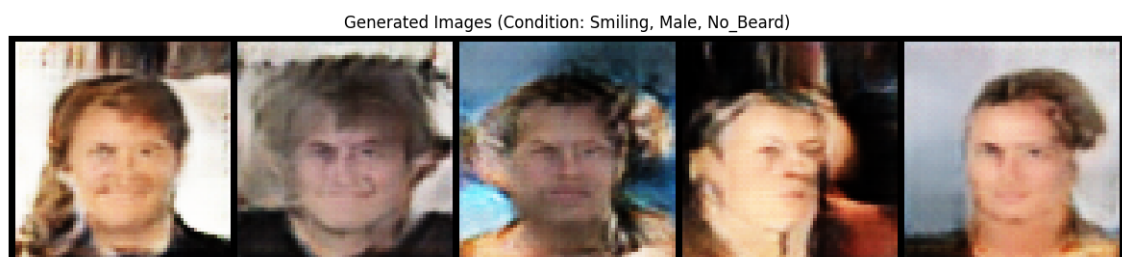


FIGURE 6 – Exemple avec attributs : Smiling, Male et No_Beard



FIGURE 7 – Exemple avec attribut Blond_Hair

Les résultats que l'on obtient sont prometteur, le modèle généralise suffisamment pour générer à chaque fois des visages différents et dans le cas d'un seul attribut : Blond_Hair, on devine en effet l'attribut sur les images.

Pour tester la diversité du modèle et l'évaluer, on peut utiliser **LPIPS**.

3.2.2 LPIPS

Pour comparer les images de manière plus formelle, on peut utiliser **LPIPS**.

Au lieu de comparer les pixels bruts, LPIPS (Learned Perceptual Image Patch Similarity) utilise un réseau de neurones pré-entraîné (généralement VGG16 ou AlexNet, entraîné sur des millions d'images réelles avec ImageNet) comme "oeil humain" pour comparer les images.

LPIPS suit le processus suivant :

- **Extraction des caractéristiques** : Les deux images à comparer passent à travers le réseau VGG.
- **Récupération des activations** : LPIPS ne regarde pas la classification finale du réseau, mais s'arrête aux couches intermédiaires (les couches de convolution). Il extrait les *feature maps*, qui représentent les textures, les formes et les couleurs perçues par le réseau (espace latent).
- **Calcul de la distance** : La métrique calcule la distance euclidienne entre ces cartes de caractéristiques pour chaque couche l , souvent pondérée par un vecteur w

Le calcul de la distance se fait selon la formule suivante :

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\hat{y}_{hw}^l - \hat{y}_{0hw}^l)\|_2^2$$

où $\hat{y}$ représente les activations du réseau pour l'image x , et $\hat{y}_0$ les activations pour l'image x_0 .

```

1 def compute_generation_diversity(generator, base_condition, device, num_pairs=10):
2     generator.eval()
3     lpips_metric = LearnedPerceptualImagePatchSimilarity(net_type='vgg').to(device)
4
5     lpips_scores = []
6
7     with torch.no_grad():
8         for _ in range(num_pairs):
9             # generate two different images using the exact same condition
10            noise1 = torch.randn(1, Z_DIM, 1, 1, device=device)
11            noise2 = torch.randn(1, Z_DIM, 1, 1, device=device)
12
13            # we need batch size 1 for the condition here
14            cond = base_condition[0].unsqueeze(0)
15
16            img1 = generator(noise1, cond)
17            img2 = generator(noise2, cond)
18
19            # LPIPS expects images in range [-1, 1], which matches our generator
20            output
21            score = lpips_metric(img1, img2)
22            lpips_scores.append(score.item())
23
24            avg_lpips = sum(lpips_scores) / len(lpips_scores)
25            print(f"Average LPIPS Diversity Score (Higher is better): {avg_lpips:.4f}")

```

Listing 7 – Algo de score LPIPS

Contrairement à la précision (où plus haut = meilleur), LPIPS est une mesure de distance :

LPIPS proche de 0 : Les deux images sont perceptuellement identiques.

LPIPS élevé (ex : > 0.5) : Les deux images sont structurellement et visuellement très différentes.

Dans le cas d'un CGAN, nous n'avons pas d'image "cible" exacte pour calculer une erreur. Si on demande au modèle de générer "un homme à lunettes", il n'y a pas une seule bonne réponse, il en existe des millions.

Nous allons donc faire un test de diversité : nous donnons au Générateur la "même condition" (ex : "Femme, Souriante, Jeune") mais avec deux vecteurs de bruit différents (z_1 et z_2). Le modèle doit générer deux femmes différentes qui respectent ces critères. Nous calculons le score LPIPS entre ces deux images générées.

Si le LPIPS est très bas, cela signifie que le GAN a triché. Il a mémorisé un seul visage de "Femme souriante" et le recrache en boucle peu importe le bruit z .

Si le LPIPS est suffisamment élevé, cela prouve que le Générateur a bien appris la distribution globale et est capable d'inventer des visages variés et uniques pour une même condition.

Dans notre contexte d'évaluation de la diversité interne d'un GAN, un score LPIPS plus élevé indique une bonne performance.

Pour l'entraînement sur 5 époques, on obtient un score de 0.5899.

3.2.3 Entraînement 30 époques

Dans l'article classique du DCGAN ([1]), il est conseillé de faire un entraînement de 20 à 50 époques, je vais donc tester un entraînement sur 30 époques.

```
1 gen_30 = Generator(Z_DIM, C_DIM, FEATURES_GEN).to(device)
2 disc_30 = Discriminator(C_DIM, FEATURES_DISC).to(device)
3
4 NUM_EPOCHS = 30
5
6 G_losses_30, D_losses_30 = train_cgan(gen_30, disc_30, dataloader, device,
    num_epochs=NUM_EPOCHS, z_dim=Z_DIM)
```

Listing 8 – Entraînement du CGAN sur 30 époques

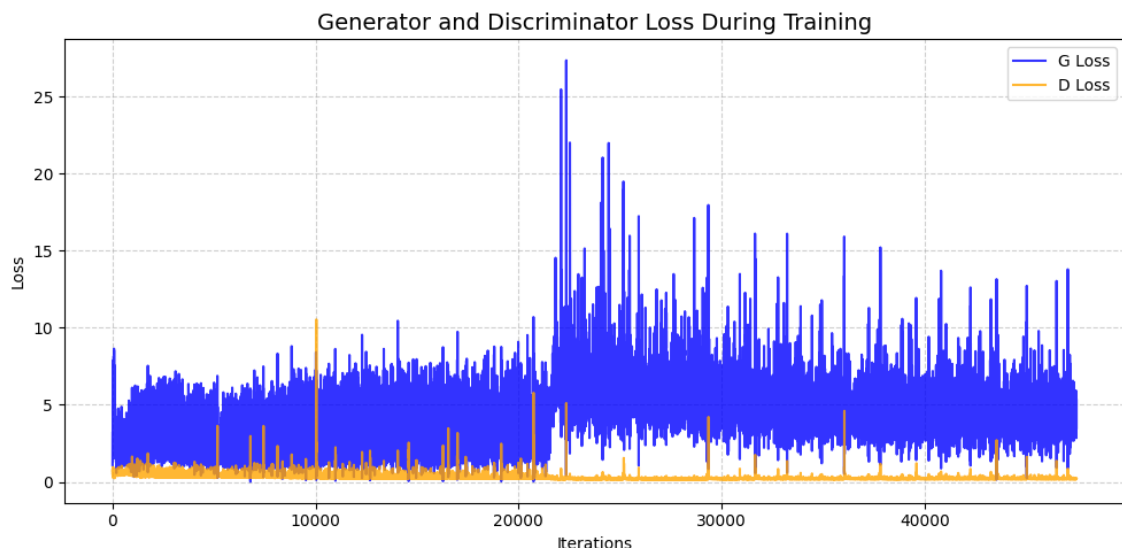


FIGURE 8 – Courbes de Loss du CGAN sur 30 époques

On remarque que passé un certain nombre d'époques, le modèle commence à diverger, on a un saut dans la loss. C'est le signe d'un sur-apprentissage ou d'un *Gradient Vanishing*. En effet, la

courbe du discriminateur converge très vite et se stabilise alors que celle du générateur explose car le modèle n'apprend plus (le gradient tend vers 0).

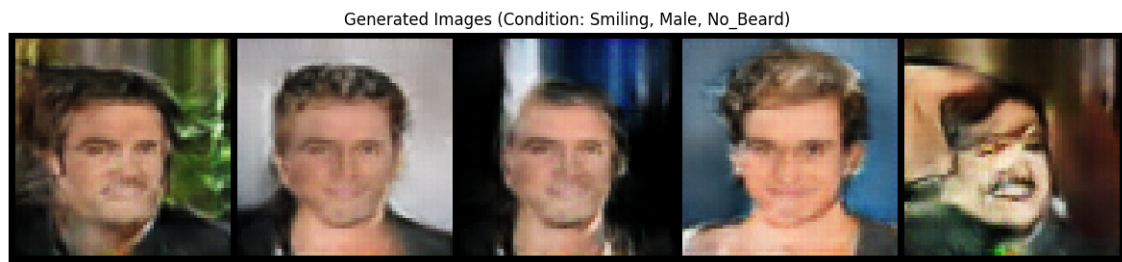


FIGURE 9 – Exemple avec attributs : Smiling, Male et No_Beard

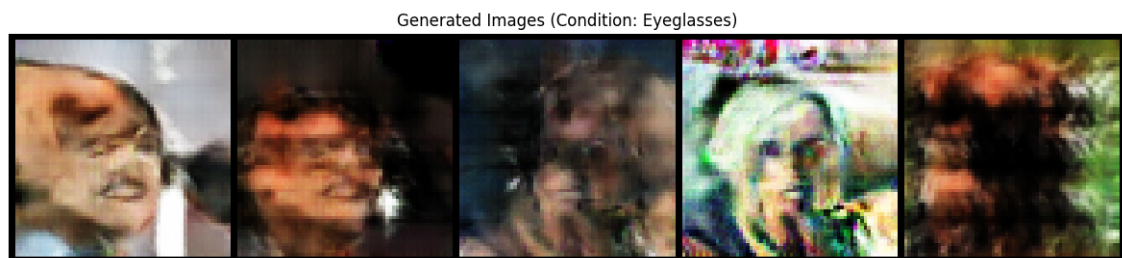


FIGURE 10 – Exemple avec attribut Eyeglasses

Sur la classe **Eyeglasses** qui est peu représentée dans le dataset, on obtient ici des images "glitchées" à cause du sur-apprentissage que l'on ne peut pas utiliser.

On remarque par ailleurs sur 9 que l'on a perdu en généralité, on reconnaît sur les 4 premières images la même personne et le 5ème avec la moustache apparaît aussi dans 10.

On obtient un score LPIPS de 0.4689 donc un a perdu en diversité par rapport

3.2.4 Entraînement 15 époques (modèle final)

On a vu qu'avec 30 époques, on a du sur-apprentissage, le modèle n'arrive plus à généraliser (diversité) et les images sont "glitchées" en raison de la répartition très inégale dans le dataset.

On va donc essayer de faire un entraînement sur 15 époques.

```
1 gen_15 = Generator(Z_DIM, C_DIM, FEATURES_GEN).to(device)
2 disc_15 = Discriminator(C_DIM, FEATURES_DISC).to(device)
3
4 NUM_EPOCHS = 15
5
6 G_losses_15, D_losses_15 = train_cgan(gen_15, disc_15, dataloader, device,
    num_epochs=NUM_EPOCHS, z_dim=Z_DIM)
```

Listing 9 – Entraînement du CGAN sur 15 époques

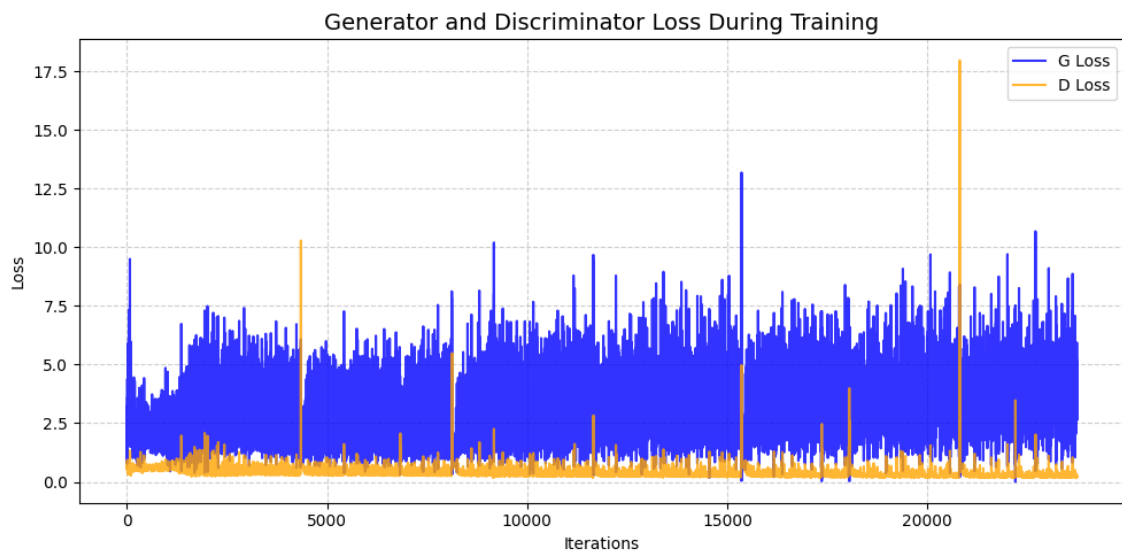


FIGURE 11 – Courbes de Loss du CGAN sur 15 époques

Contrairement à l'entraînement sur 30 époque, ici la loss ne diverge pas (8).

Cependant, on remarque que le discriminateur converge plus vite que le générateur ce qui expliquait le phénomène sur 30 époques, en effet, le discriminateur est trop fort par rapport au générateur donc le générateur n'apprend plus car son gradient tend vers 0. Par ailleurs, la courbe d'entraînement du générateur est souvent le signe d'un learning rate trop grand ce qui entraîne cette instabilité, mais j'ai essayé de le diminuer rien n'y fait.

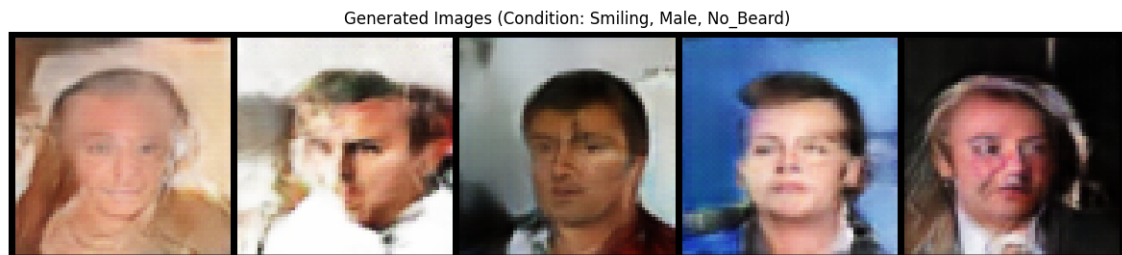


FIGURE 12 – Exemple avec attributs : Smiling, Male et No_Beard

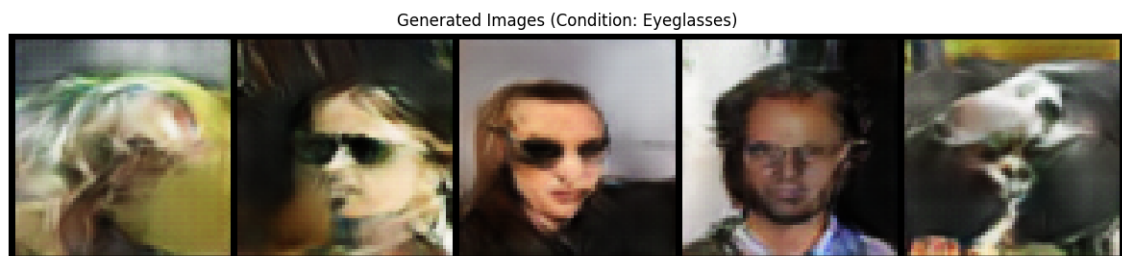


FIGURE 13 – Exemple avec attribut Eyeglasses

On remarque que l'on garde de la généralité et le modèle est plutôt efficace, même sur un attribut relativement peu représenté : **Eyeglasses** 13 (à seulement 20%) on devine la présence de lunettes sur les images (images 2, 3 et 4) malgré quelques images "glitchées".

On obtient un score LPIPS de 0.5865 donc similaire à l'entraînement sur 5 époques.

Les entraînements sont très longs (plusieurs heures), on ne va donc pas tester toutes les combinaisons d’hyperparamètres possibles. Etant donné que l’on a obtenu de bons résultats ici avec 15 époques, on va finalement garder ce modèle pour notre application cf. 6.

3.2.5 Pistes d’améliorations

Il pourrait être intéressant de faire de l’optimisation d’hyperparamètres avec un framework tel Optuna sur le modèle pour tenter de l’améliorer mais cela prendrait des jours et on se heurte à un problème quand il s’agit des modèles génératifs : quelle métrique à minimiser/optimiser choisir ? En effet, puisque la Loss du Générateur et du Discriminateur jouent au chat et à la souris, une Loss basse pour l’un signifie une Loss haute pour l’autre, dans un environnement de recherche idéal, Optuna générerait des milliers d’images à la fin de chaque essai, calculerait le score FID (Fréchet Inception Distance), et chercherait à le minimiser. Cependant, cela prendrait des jours de calcul. C’est pour cela que l’on utilise déjà les valeurs “`optim.Adam(gen.parameters(), lr=0.0002, betas=(0.5, 0.999))`” qui sont celles qui ont données les meilleurs résultats dans les papiers de recherche sur le CGAN.

De plus, en remplaçant le bruit d’entrée du générateur une fois entraîné par un visage déterminé, on pourrait essayer de faire générer au modèle des images avec les attributs que l’on souhaite, par exemple prendre son propre visage et y ajouter des lunettes.

On pourrait aussi explorer du côté des **StyleGAN** qui utilise un genre de MLP en entrée du générateur pour structurer et contrôler le bruit latent du vecteur d’entrée z . C’est un également un modèle un peut à l’image des **Progressive Growing GAN** qui s’entraîne en augmentant la dimension des images d’entrée ($4x4$ puis $8x8$, etc.) et la profondeur du Générateur/Discriminateur pour conserver en stabilité tout au long de l’entraînement.

4 Le modèle CVAE

Nous avons décidé d'utiliser un CVAE (Conditional Variational Autoencoder). Il s'agit d'une extension du VAE classique intégrant des conditions externes — dans notre cas, les vecteurs d'attributs faciaux (labels) — à la fois à l'entrée de l'encodeur et du décodeur. Cette architecture permet au modèle d'apprendre la corrélation explicite entre l'espace latent et des caractéristiques visuelles spécifiques, offrant ainsi un contrôle sur la génération (par exemple, l'ajout de lunettes). Cependant, bien que stables à l'entraînement, les CVAE présentent des limites structurelles et mathématiques fondamentales pour la génération d'images haute résolution :

- **Le flou inhérent à la fonction de perte (Blurriness)** : L'entraînement d'un VAE repose traditionnellement sur la minimisation de l'erreur de reconstruction pixel par pixel (souvent l'erreur quadratique moyenne, ou norme L_2). Face à l'incertitude spatiale (par exemple, le placement exact des contours d'une monture de lunettes ou la texture détaillée des cheveux), le modèle opte mathématiquement pour la moyenne des valeurs possibles pour minimiser son erreur. Ce nivellement se traduit visuellement par des images floues et un manque de netteté dans les hautes fréquences.
- **Le compromis de la divergence KL** : La fonction de perte inclut une pénalité (la divergence de Kullback-Leibler) qui force l'espace latent à suivre une distribution normale standard $\mathcal{N}(0, 1)$. Si cette contrainte est trop forte, le modèle risque de trop lisser les données et d'ignorer la condition fournie (un phénomène appelé posterior collapse).
- **L'enchevêtrement des caractéristiques (Entanglement)** : Malgré l'ajout de la condition, le CVAE peine parfois à isoler parfaitement un attribut. Dans un visage, des caractéristiques comme « porter des lunettes », « âge » et « forme des yeux » sont fortement corrélées. Modifier la condition pour ajouter des lunettes entraîne souvent des modifications indésirables sur le reste du visage, car l'espace latent n'est pas parfaitement démêlé.

C'est pour pallier ces limites, et particulièrement pour forcer le modèle à retenir davantage de détails haute fréquence, que nous avons conçu une architecture profonde. Nous avons utilisé un CVAE totalisant 48 176 771 paramètres, doté d'un espace latent de dimension 512, et entraîné sur 100 époques. Pour prévenir le surapprentissage (overfitting) tout en préservant l'alignement strict du jeu de données CelebA, nous avons appliqué une augmentation de données restreinte au retournement horizontal (random horizontal flip). Ce modèle constitue le meilleur compromis observé entre temps de calcul et qualité de reconstruction visuelle, après plusieurs itérations sur des architectures plus légères.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class CVAE(nn.Module):
6     def __init__(self, latent_dim=512, num_classes=40):
7         super(CVAE, self).__init__()
8         self.num_classes = num_classes
9
10        self.enc_conv1 = nn.Conv2d(3 + num_classes, 64, kernel_size=4, stride=2,
padding=1)
11        self.enc_bn1 = nn.BatchNorm2d(64)
12        self.enc_conv2 = nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1)
13        self.enc_bn2 = nn.BatchNorm2d(128)
14        self.enc_conv3 = nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1)
15        self.enc_bn3 = nn.BatchNorm2d(256)
16        self.enc_conv4 = nn.Conv2d(256, 512, kernel_size=4, stride=2, padding=1)
17        self.enc_bn4 = nn.BatchNorm2d(512)
18        self.enc_conv5 = nn.Conv2d(512, 1024, kernel_size=4, stride=2, padding=1)
19        self.enc_bn5 = nn.BatchNorm2d(1024)
20
21        self.fc_mu = nn.Linear(1024 * 4 * 4, latent_dim)
22        self.fc_logvar = nn.Linear(1024 * 4 * 4, latent_dim)
23
24        self.dec_fc = nn.Linear(latent_dim + num_classes, 1024 * 4 * 4)
25
26        self.dec_conv0 = nn.ConvTranspose2d(1024, 512, kernel_size=4, stride=2,
padding=1)
27        self.dec_bn0 = nn.BatchNorm2d(512)
28        self.dec_conv1 = nn.ConvTranspose2d(512, 256, kernel_size=4, stride=2,
padding=1)
29        self.dec_bn1 = nn.BatchNorm2d(256)
30        self.dec_conv2 = nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2,
padding=1)
```

```

31     self.dec_bn2 = nn.BatchNorm2d(128)
32     self.dec_conv3 = nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2,
padding=1)
33     self.dec_bn3 = nn.BatchNorm2d(64)
34     self.dec_conv4 = nn.ConvTranspose2d(64, 3, kernel_size=4, stride=2, padding
=1)
35
36     def encode(self, x, c):
37         batch_size, _, h, w = x.size()
38         c_expanded = c.view(batch_size, self.num_classes, 1, 1).expand(-1, -1, h, w
)
39
40         x = torch.cat([x, c_expanded], dim=1)
41
42         x = F.leaky_relu(self.enc_bn1(self.enc_conv1(x)), 0.2)
43         x = F.leaky_relu(self.enc_bn2(self.enc_conv2(x)), 0.2)
44         x = F.leaky_relu(self.enc_bn3(self.enc_conv3(x)), 0.2)
45         x = F.leaky_relu(self.enc_bn4(self.enc_conv4(x)), 0.2)
46         x = F.leaky_relu(self.enc_bn5(self.enc_conv5(x)), 0.2)
47         x = x.view(x.size(0), -1)
48
49         mu = self.fc_mu(x)
50         logvar = self.fc_logvar(x)
51         return mu, logvar
52
53     def reparameterize(self, mu, logvar):
54         std = torch.exp(0.5 * logvar)
55         eps = torch.randn_like(std)
56         return mu + eps * std
57
58     def decode(self, z, c):
59         z = torch.cat([z, c], dim=1)
60         z = self.dec_fc(z)
61         z = z.view(z.size(0), 1024, 4, 4)
62         z = F.relu(self.dec_bn0(self.dec_conv0(z)))
63         z = F.relu(self.dec_bn1(self.dec_conv1(z)))
64         z = F.relu(self.dec_bn2(self.dec_conv2(z)))
65         z = F.relu(self.dec_bn3(self.dec_conv3(z)))
66         z = torch.sigmoid(self.dec_conv4(z))
67         return z
68
69     def forward(self, x, c):
70         mu, logvar = self.encode(x, c)
71         z = self.reparameterize(mu, logvar)
72         x_recon = self.decode(z, c)
73         return x_recon, mu, logvar
74
75 loss_fn = nn.MSELoss()
76
77 def train(dataloader, model, optimizer, device, beta=1.0):
78     size = len(dataloader.dataset)
79     model.train()
80     for batch, (X, y) in enumerate(dataloader):
81         X = X.to(device)
82         y = y.to(device)
83
84         optimizer.zero_grad()
85
86         pred, mu, log_sigma = model(X, y)
87         reconstruction_loss = F.mse_loss(pred, X, reduction='sum')
88         kl = -0.5 * torch.sum(1 + log_sigma - mu.pow(2) - log_sigma.exp())
89         loss = reconstruction_loss + beta * kl
90
91         loss.backward()
92         optimizer.step()
93
94     if batch % 100 == 0:
95         loss_val, current = loss.item(), (batch + 1) * len(X)
96         print(f"loss: {loss_val:>7f} [{current:>5d}/{size:>5d}]")

```

Listing 10 – Code CVAE

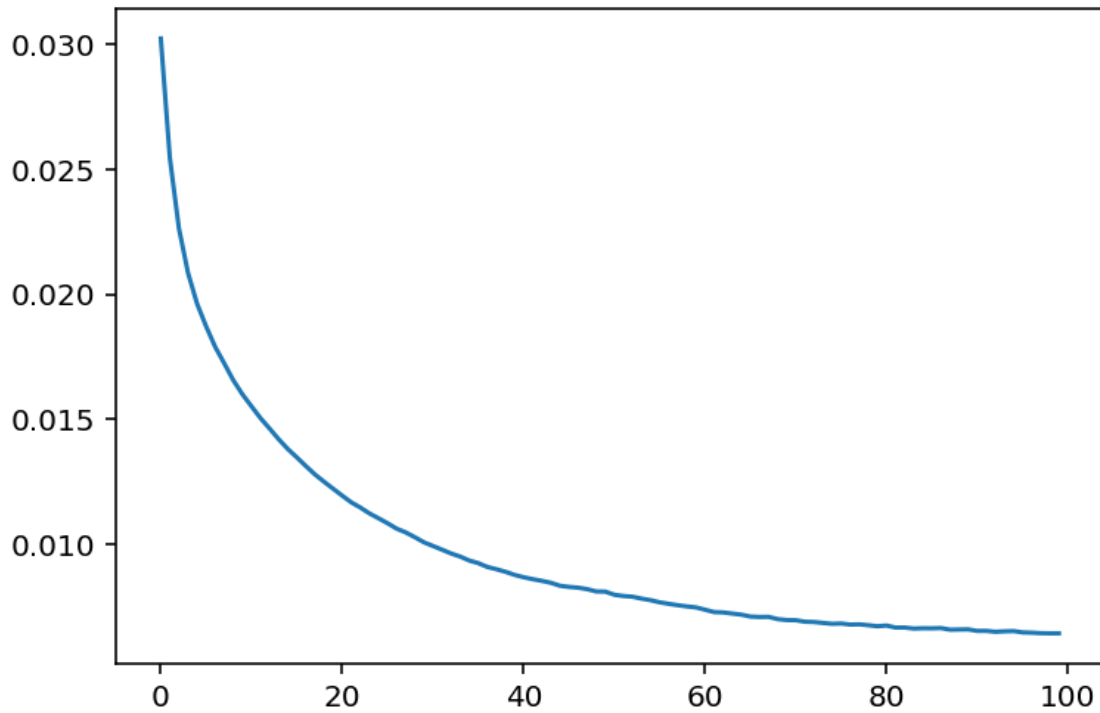


FIGURE 14 – evolution de la loss de la validation du cvae

4.1 Résultats CVAE

Pour évaluer les capacités de notre modèle, nous avons procédé à trois tests distincts ciblant la reconstruction, l'édition d'attributs et la génération.

4.1.1 Reconstruction images dataset test

Ce premier test évalue la capacité de l'encodeur à compresser l'information et celle du décodeur à la restituer fidèlement, en fournissant au modèle l'image originale ainsi que son vecteur de labels exact.

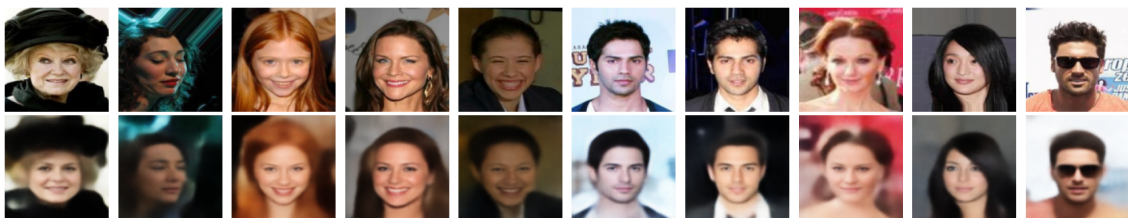


FIGURE 15 – Reconstitution test

Les résultats de la Figure 15 démontrent une reconstruction performante. L'identité globale du sujet est préservée, les attributs majeurs (chapeaux, type de chevelure, morphologie) sont respectés, et les arrière-plans sont fidèlement restitués. Le modèle a donc appris une représentation latente riche, bien que la texture globale souffre d'un léger flou caractéristique évoqué précédemment.

4.1.2 ajout features images dataset train

L'un des avantages du CVAE réside dans sa capacité théorique à éditer une image. Nous avons encodé une image de l'ensemble d'entraînement pour obtenir son vecteur latent, puis nous l'avons décodée en modifiant artificiellement un seul attribut cible dans le vecteur de condition.

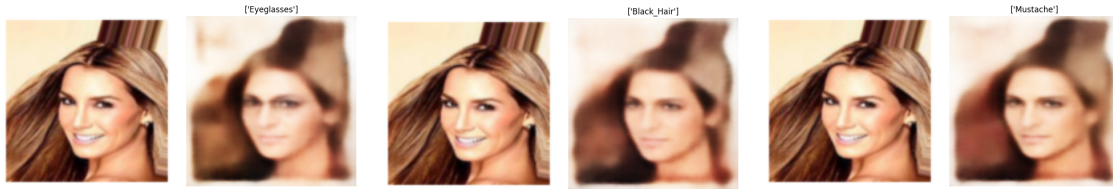


FIGURE 16 – Ajout lunettes

FIGURE 17 – Cheveux noirs

FIGURE 18 – Ajout moustache

FIGURE 19 – Comparaison des modifications d'attributs sur une même image source.

Les résultats mettent en évidence le phénomène d'enchevêtrement (entanglement). L'ajout de lunettes est géré de manière satisfaisante, bien que les contours manquent de netteté. En revanche, le modèle échoue à isoler correctement le changement de couleur de cheveux vers le noir, ou l'ajout d'une moustache. Peut être que le réseau associe fortement ces attributs au genre masculin dans sa représentation globale, et refuse d'appliquer des modifications locales fortes contredisant la structure faciale encodée.

4.1.3 Génération aléatoire avec feature

Dans ce dernier test, nous évaluons la génération ex nihilo. Nous échantillonnons des vecteurs de bruit purement aléatoires depuis une distribution normale $\mathcal{N}(0,1)$ pour l'espace latent z , et nous les passons au décodeur avec un vecteur de condition c contenant uniquement des zéros, à l'exception d'un unique « 1 » pour l'attribut désiré.

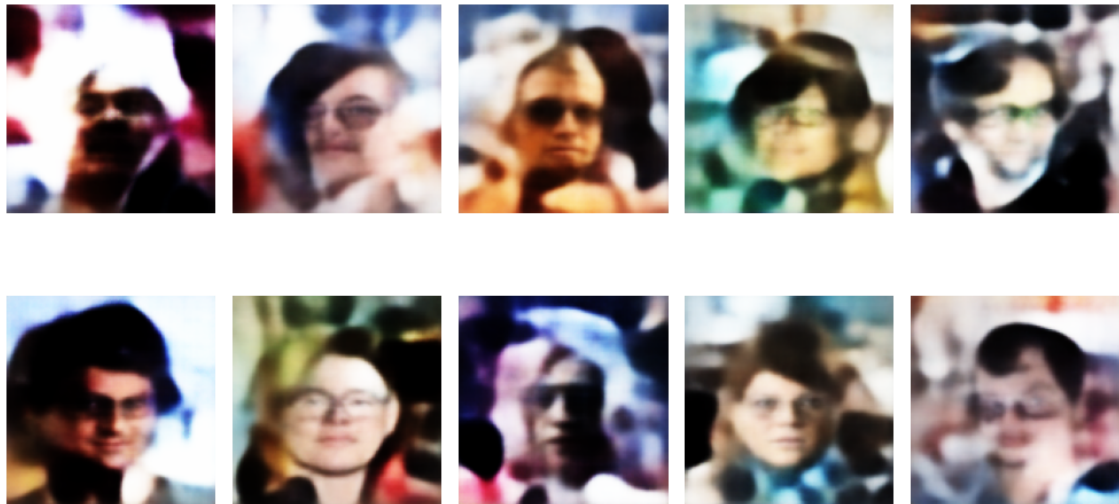


FIGURE 20 – Générations aléatoires avec lunettes

Comme observé sur la Figure ??, le CVAE a intégré la structure d'un visage aligné (position des yeux, nez, bouche au centre de l'image), le fond par contre est complètement chaotique, dû à des calculs de "moyenne" des vecteurs continus. La condition est globalement respectée : des lunettes de soleil opaques apparaissent nettement, tandis que les lunettes de vue génèrent des structures plus sombres ou floues autour des yeux, traduisant l'incertitude du réseau face à la finesse de la monture.

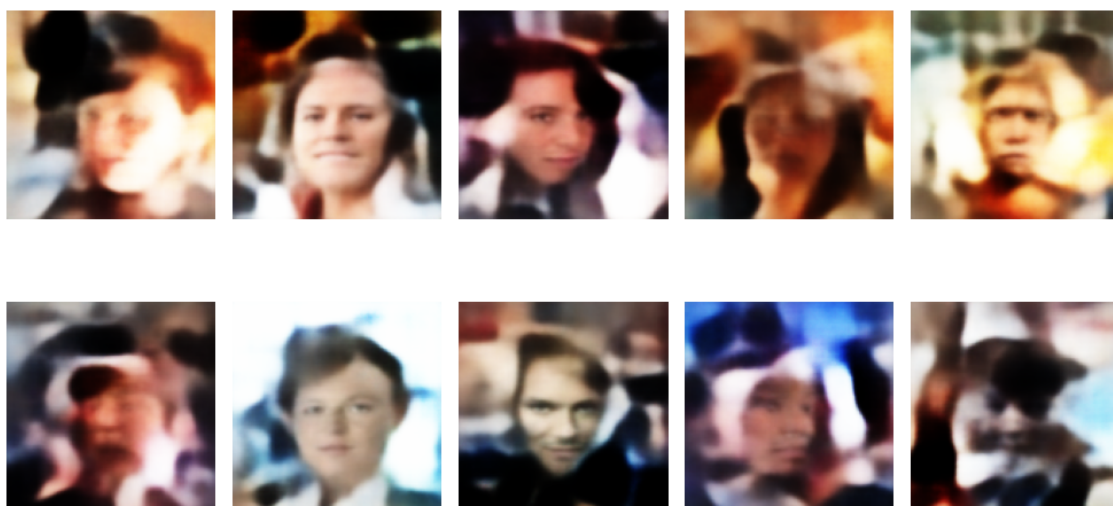
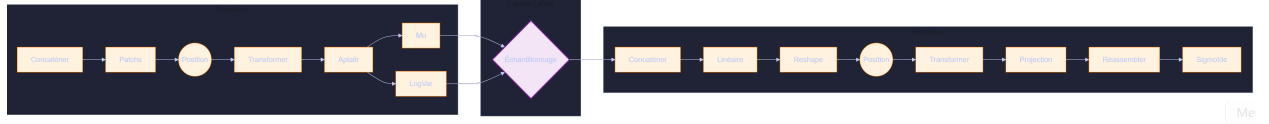


FIGURE 21 – Générations aléatoires chauve

Le conditionnement sur l'attribut « Chauve » (Figure ??) s'avère plus complexe à interpréter pour le modèle. Les résultats sont mitigés : la génération produit fréquemment un compromis structurel, se traduisant par des sujets partiellement chauves, des lignes de cheveux fuyantes, ou des crânes lissés mais entourés d'ombres ressemblant à des cheveux. Ce lissage est un symptôme classique de la perte MSE cherchant à moyenner les zones à forte variance texturale.

5 Modèle exploratoire : CVAE-Transformer (ViT-CVAE)

Dans une démarche d'exploration de l'état de l'art, nous avons cherché à intégrer l'architecture Transformer, actuellement dominante dans les tâches de génération et de transformation d'images. Sachant que l'entraînement d'un Transformer génératif pur (de type autoregressif ou diffusion) requiert des ressources de calcul hors de portée de notre matériel, nous avons opté pour une architecture hybride : un CVAE dont les réseaux convolutifs ont été remplacés par des couches d'attention (Vision Transformer). Le mécanisme d'encodage divise l'image en une séquence de patches (8×8 pixels), traités par un Transformer Encoder pour générer l'espace latent. Le décodeur utilise un Transformer Decoder pour reformer les patches avant de les réassembler spatialement.



L'entraînement de ce modèle s'est avéré particulièrement complexe et laborieux.

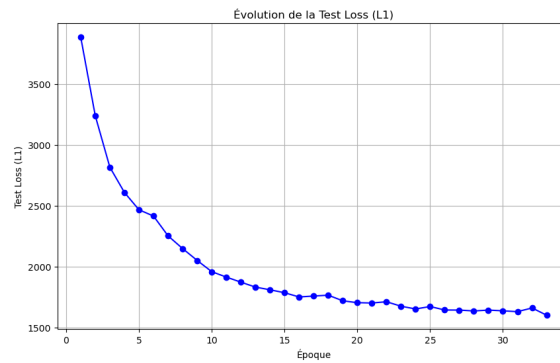


FIGURE 22 – Évolution de la fonction de perte du ViT-CVAE

5.1 Résultats visuels et limites

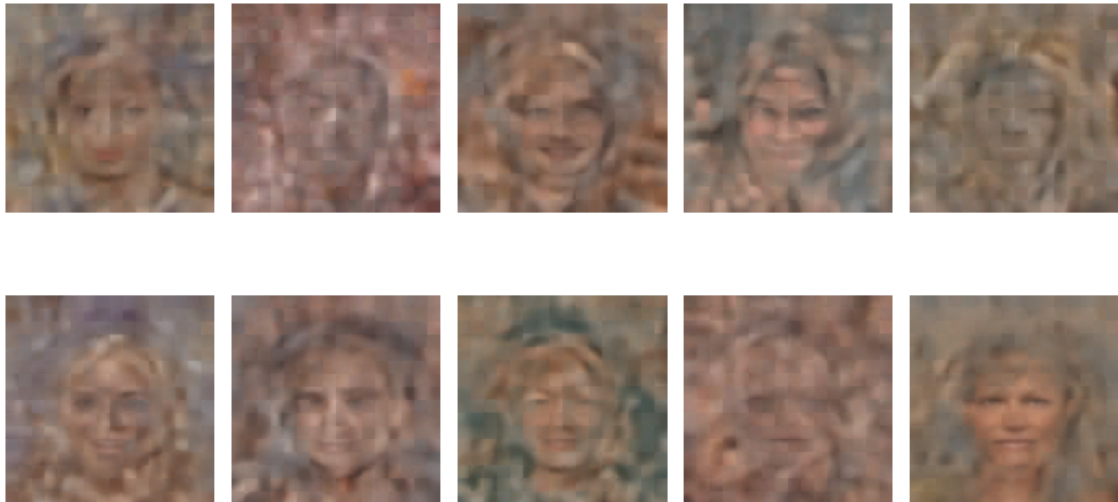


FIGURE 23 – génération aléatoire attribut « Lunettes ».

Les résultats en génération sont visuellement inexploitable. L'image souffre d'un violent effet de mosaïque : le modèle peine à assurer une continuité spatiale entre les patches de 8×8 . La séparation entre l'arrière-plan et le sujet est inexistante, et les lunettes ne sont pas générées.

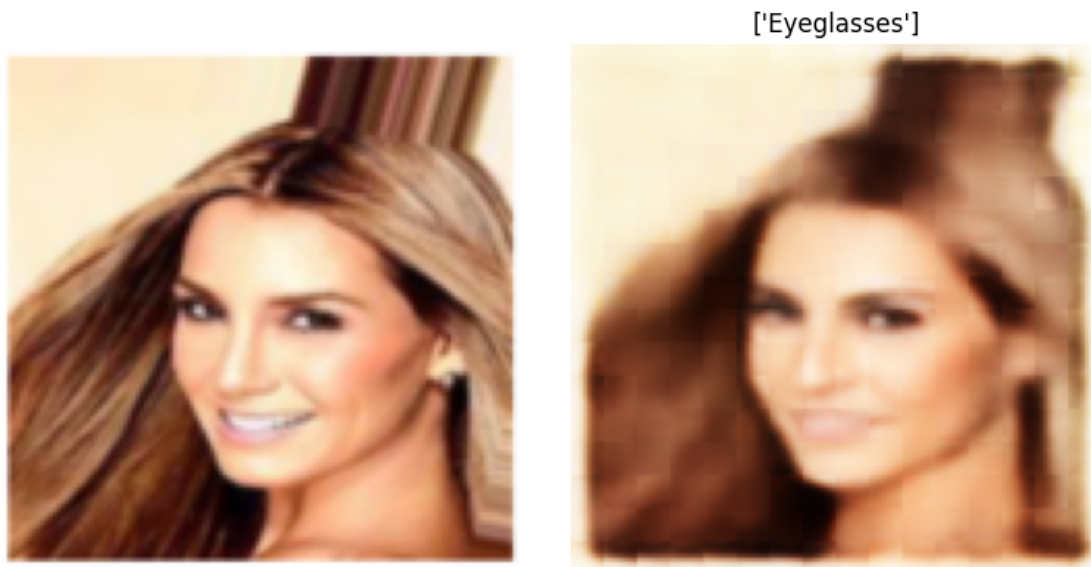


FIGURE 24 – ajout de lunettes sur une image de train

Lors de la tâche de transformation, le modèle parvient à reconstituer la colorimétrie globale et la géométrie de base du visage source. Toutefois, le réseau échoue complètement à appliquer la modification demandée (l'ajout de lunettes).

Conclusion sur l'approche Transformer : Cet échec relatif s'explique par deux facteurs fondamentaux :

- L'absence de biais inductif : Contrairement aux couches convolutives qui comprennent nativement que des pixels voisins forment des structures continues, le Transformer traite chaque patch de manière indépendante. Il doit apprendre la topologie de l'image de zéro, ce qui nécessite des ensembles de données gigantesques.
- Le sous-entraînement face à la complexité matérielle : Pour qu'un ViT génère des résultats cohérents sans pré-entraînement, il requiert des milliers d'époques d'apprentissage (avec une protection contre l'explosion du gradient). Nos contraintes matérielles et temporelles rendent cette convergence impossible.

Il n'est pas utile de rajouter ce modèle à notre interface web car c'est un échec pour la génération d'images.

6 Interface pour notre application (Inférence)

Pour pouvoir faire facilement de l'inférence et tester notre projet, nous avons fait une interface streamlit dans `streamlit run app/app.py`.

Dans l'onglet GAN, on peut choisir les attributs que l'on veut voir apparaître sur les visages générés.



FIGURE 25 – Interface streamlit GAN permettant de cocher les attributs, exemple avec Eyeglasses

Ensuite, on peut choisir le nombre de visages à générer et les générer, le modèle utilise le fichier `GAN/saved_models/cgan_generator_15.pth` qui contient les poids du modèle entraîné précédemment.

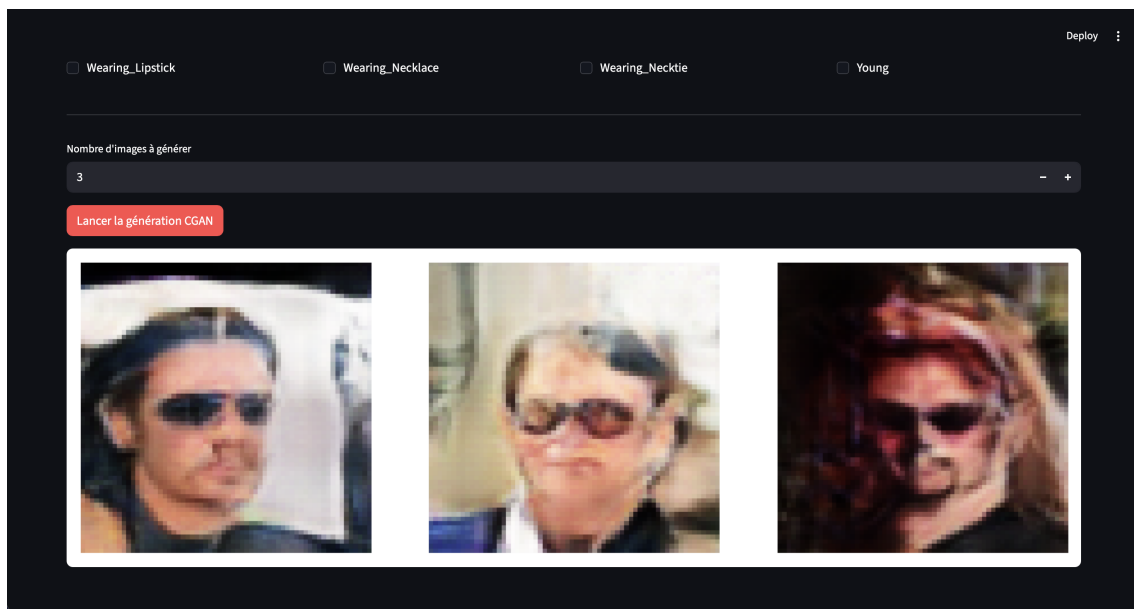


FIGURE 26 – Interface streamlit GAN, génération de 3 visages

Cela donne un exemple d'interface facilement utilisable pour notre projet.
De même pour le CVAE

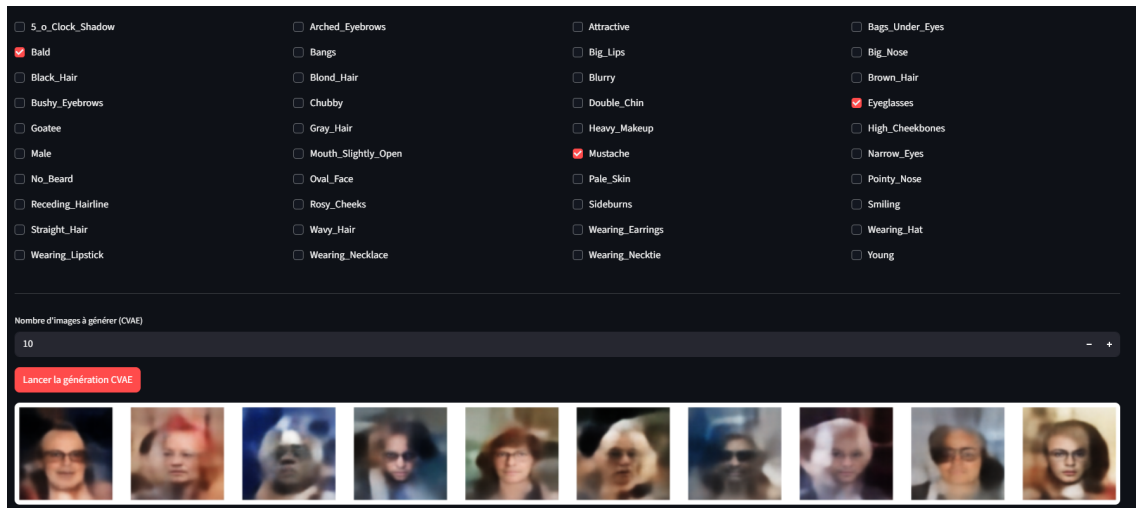


FIGURE 27 – Interface streamlit CVAE, génération de 10 visages

7 Conclusion

8 Bibliographie

Références

- [1] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv*, 2015. <https://arxiv.org/abs/1511.06434>.